ÉCOLE NATIONALE D'INGÉNIEURS DE CARTHAGE · ATELIER TEST LOGICIEL

Plateforme de gestion des clubs de l'ENICarthage

Rapport d'atelier — exigences, plan de test, exécution et recette

20	32	116	22	18
exigences	cas de test	tests Jest	tests E2E (2 navigateurs)	anomalies détectées

Réalisé par : Mohamed Aziz Rouissi · Mohamed Bouchahda · Mohamed Hosni Missaoui

Encadrement : [Nom de l'encadrant]

Outil de test : Jest 30 (+ React Testing Library) · Playwright · k6 · JaCoCo · ESLint · PMD

Gestion des tests et des exigences : GitHub Issues, jalons et Pull Requests — github.com/azizrouissi19-code/enicar-club-management-qa

Date : 11 novembre 2026 · Année universitaire 2026 – 2027

Sommaire

1. Introduction et objectifs des ateliers

2. Choix du projet (objectif 1)

3. Installation de l'environnement de test

4. Besoins fonctionnels et non fonctionnels (objectif 2)

5. Outil de gestion des tests et des exigences : GitHub (objectif 3)

6. Plan de test (objectif 4)

7. Exécution du plan de test (objectif 5)

7.1 Tests statiques · 7.2 Unitaires (Jest) · 7.3 Intégration (API) · 7.4 Structurels (couverture)

7.5 Système (E2E) · 7.6 Acceptation · 7.7 Non fonctionnels · 7.8 Changements et maintenance · 7.9 Suivi sur GitHub

8. Bilan : anomalies, métriques et verdict de recette

9. Conclusion

10. Annexe : rejouer les tests

1. Introduction et objectifs des ateliers

Ce rapport présente la démarche de test menée sur notre plateforme de gestion des clubs de l'ENICarthage. Il suit les cinq objectifs fixés par l'atelier. Tous les résultats présentés proviennent d'exécutions réelles ; les fichiers bruts sont conservés dans le dossier `qa/results` du dépôt.

Objectif	Travail réalisé	Chap.
1. Choisir un projet	Notre plateforme de gestion des clubs (Spring Boot + React + MySQL), lancée en local	2, 3
2. Présenter les besoins fonctionnels et non fonctionnels	10 exigences fonctionnelles et 10 non fonctionnelles, identifiées et classées par criticité	4
3. Utiliser un outil de gestion de test (et des exigences)	GitHub : 70 issues liées (exigences, cas de test, anomalies), libellés, 2 jalons (campagnes), Pull Request de correction ; matrice de traçabilité	5
4. Élaborer un plan de test	Plan IEEE 829 : 4 niveaux (unitaire, intégration, système, acceptation), 5 types (fonctionnel, non fonctionnel, structurel, lié aux changements, maintenance), stratégie basée sur les risques	6
5. Exécuter le plan de test	Techniques statiques et dynamiques, tests manuels et automatisés, 2 itérations ; 18 anomalies dont 15 corrigées et re-testées	7, 8

94/138
tests réussis — itération 1

136/138
tests réussis — itération 2

19/20
exigences satisfaites (dont 3 avec réserve)

15/18
anomalies corrigées

2. Choix du projet (objectif 1)

2.1 Présentation

La plateforme centralise la vie associative de l'école : clubs, événements, salles, équipements et réservations de ressources. Elle repose sur trois profils :

Profil	Rôle principal
Administration (`ADMINISTRATOR`)	Crée les clubs et le compte de leur président, gère les salles et les équipements, valide ou rejette les réservations
Président de club (`CLUB_PRESIDENT`)	Organise les événements de son club
Membre de club (`CLUB_MEMBER`)	S'inscrit, consulte les clubs et événements, réserve des ressources

2.2 Version testée

Le dépôt GitHub de l'équipe ne contenait qu'une partie du frontend (fichiers vides ou tronqués, aucune trace du backend). La version complète a été récupérée depuis le Google Drive de l'équipe, versionnée et étiquetée **v1.0.0** : c'est la version testée lors de l'itération 1. Les correctifs forment la version **v1.0.1** (itération 2). Le tout est publié dans un nouveau dépôt : github.com/azizrouissi19-code/enicar-club-management-qa.

2.3 Pourquoi ce projet se prête au test

- **Application complète :** interface React, API REST Spring Boot, base MySQL : tous les niveaux de test s'appliquent.
- **Trois rôles :** terrain idéal pour les tests de contrôle d'accès (tables de décision rôle × action).
- **Règles métier testables :** capacité d'une salle (valeurs limites), cycle de vie d'une réservation (transitions d'états), adresse institutionnelle (partitions d'équivalence).
- **Données sensibles :** mots de passe et comptes, d'où des exigences de sécurité.

2.4 Architecture technique

Couche	Technologies
Frontend	React 19, Vite 8, React Router 7, Axios, Tailwind CSS
Backend	Java 17, Spring Boot 4.0, Spring Security, Spring Data JPA / Hibernate 7, Lombok, BCrypt
Base de données	MySQL (XAMPP), schéma généré par Hibernate (`ddl-auto=update`), données initiales par `DataLoader`

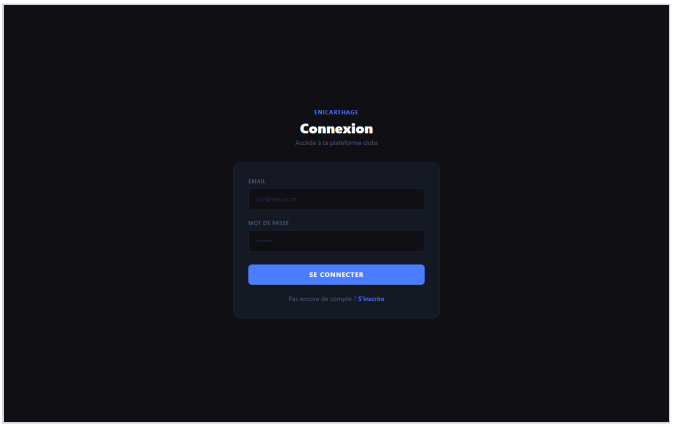


Figure 1 – Page de connexion

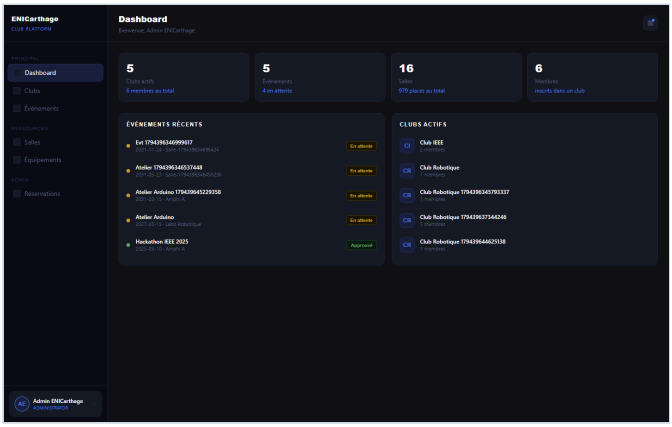


Figure 2 – Tableau de bord de l'administrateur (v1.0.1)

3. Installation de l'environnement de test

3.1 Outils

Outil	Version	Rôle
JDK / Maven Wrapper	17 · 3.9	Compilation et exécution du backend
Node.js / npm	24 · 11	Frontend, Jest, Playwright
Docker Desktop	29	MySQL 8.4, MariaDB 10.4 (maintenance), k6
Jest + React Testing Library + user-event	30 · 16	**Outil principal** : tests unitaires/composants (jsdom) et tests d'intégration de l'API (HTTP réel)
Playwright	1.x	Tests système E2E sur Microsoft Edge et Firefox ; aide à l'exécution des tests d'acceptation
k6 (grafana/k6)	Docker	Test de charge (50 utilisateurs virtuels)
JaCoCo	0.8.13	Couverture du backend (agent attaché au serveur pendant les tests)
ESLint · PMD/CPD · npm audit	9 · 7.7 · npm	Analyse statique JavaScript et Java, audit des dépendances
Git · GitHub · GitHub CLI	2.52 · gh 2.103	Gestion de version, des exigences, des tests et des anomalies

3.2 Architecture de l'environnement de test

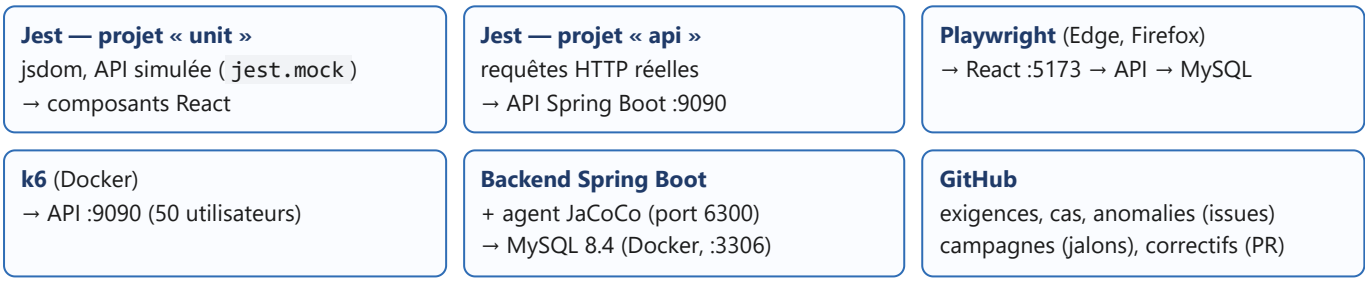


Figure 3 – Architecture de l'environnement de test

3.3 Problèmes rencontrés et solutions

Problème	Solution
Le dépôt GitHub de l'équipe était incomplet (backend absent, `ClubsPage.jsx` tronqué)	Version complète récupérée depuis le Drive, versionnée (v1.0.0) dans un nouveau dépôt
Le backend attend un MySQL XAMPP (root sans mot de passe)	MySQL 8.4 dans Docker avec la même configuration
Le projet Vite utilise des modules ES ; Jest ne les comprend pas nativement	`babel.config.cjs` (preset-env + preset-react) ; `TextEncoder` ajouté pour React Router dans jsdom
PMD ne s'exécute pas avec des règles personnalisées en ligne de commande	Profil Maven `qa` dans `pom.xml` (règles bestpractices, errorprone, security, design)
Couverture JaCoCo d'une application lancée hors Maven	Agent JaCoCo en mode `tcpserver` ; vidage avec `jacococli dump` après chaque campagne
Le jeton GitHub n'avait pas le droit `project` (tableau impossible à créer)	Droit ajouté avec `gh auth refresh -h github.com -s project` ; tableau créé par l'API GraphQL (`qa/github/project.mjs`)

4. Besoins fonctionnels et non fonctionnels (objectif 2)

Chaque exigence possède un identifiant unique, repris dans une issue GitHub (libellé `exigence`) pour la traçabilité. La criticité oriente l'effort de test (approche basée sur les risques).

4.1 Exigences fonctionnelles

Réf.	Exigence	Description	Criticité	Issue
EF-01	Authentification	Connexion par e-mail et mot de passe ; le jeton et le profil sont conservés pendant la session ; déconnexion. En cas d'échec, un message explicite reste affiché.	Critique	#1
EF-02	Inscription d'un étudiant	Un étudiant s'inscrit avec un nom, une adresse institutionnelle @enicar.tn unique et un mot de passe d'au moins 6 caractères ; il reçoit le rôle CLUB_MEMBER (le rôle envoyé par le client est ignoré).	Majeure	#2
EF-03	Rôles et contrôle d'accès	Trois rôles : ADMINISTRATOR, CLUB_PRESIDENT, CLUB_MEMBER. Toute page exige une session. Clubs, salles, équipements et validation des réservations sont réservés à l'administrateur ; la création d'événements au président (pour son club) et à l'administrateur.	Critique	#3
EF-04	Tableau de bord	Statistiques (clubs, événements, salles, membres) et listes récentes calculées à partir des données réelles de la base.	Majeure	#4
EF-05	Gestion des clubs	Consultation par tous. Création par l'administrateur avec création du compte du président (e-mail unique, nom de club obligatoire). Suppression par l'administrateur.	Critique	#5
EF-06	Gestion des événements	Le président crée un événement pour son club : titre, salle existante, date future, participants entre 0 et la capacité de la salle. Statut initial PENDING.	Critique	#6
EF-07	Gestion des salles	L'administrateur ajoute une salle : nom, capacité strictement positive, type AMPHI / SALLE / LABO.	Majeure	#7
EF-08	Gestion des équipements	L'administrateur ajoute un équipement (nom, type) ; statut initial AVAILABLE.	Mineure	#8
EF-09	Réservation de ressources	Un membre réserve une ressource existante (salle ou équipement) à une date. Pas de double réservation d'une ressource à la même date. L'administrateur approuve ou rejette (PENDING → APPROVED / REJECTED).	Critique	#9
EF-10	Profil, paramètres et notifications	Consultation du profil ; paramètres et notifications fonctionnels (pas de données fixes).	Mineure	#10

4.2 Exigences non fonctionnelles

Réf.	Exigence	Catégorie	Description (mesurable)	Criticité	Issue
ENF-01	Sécurité des identifiants	Sécurité	Mots de passe hachés (BCrypt). Le hash n'est jamais renvoyé par l'API ni stocké dans le navigateur.	Critique	#11
ENF-02	Sécurité et robustesse de l'API	Sécurité	Jeton vérifié à chaque requête ; non authentifié → 401 ; non autorisé → 403 ; entrée invalide → 4xx, jamais 500.	Critique	#12
ENF-03	Performance	Performance	95 % des requêtes en moins de 500 ms avec 50 utilisateurs simultanés.	Majeure	#13
ENF-04	Utilisabilité et affichage mobile	Utilisabilité	Messages d'erreur visibles ; utilisable sur smartphone 360 px (zone de contenu ≥ 80 % de la largeur, sans défilement horizontal).	Majeure	#14
ENF-05	Compatibilité navigateurs	Compatibilité	Fonctionnement identique sous Edge (Chromium) et Firefox.	Mineure	#15
ENF-06	Maintenabilité	Maintenabilité	0 erreur ESLint ; analyse PMD ; couverture des tests ≥ 70 % des instructions (frontend et backend).	Mineure	#16
ENF-07	Sécurité des dépendances	Sécurité	Aucune vulnérabilité connue de sévérité haute ou critique dans les dépendances de production.	Majeure	#17

ENF-08	Persistance des données	Fiabilité	Les données sont conservées après un redémarrage du backend.	Majeure	#18
ENF-09	Configuration sécurisée	Sécurité	Secrets hors du code source ; CORS limité au frontend ; pas de journalisation SQL en production.	Majeure	#19
ENF-10	Portabilité de la base de données	Portabilité	Fonctionne avec MySQL 8 et avec MariaDB (XAMPP), sans modification du code.	Mineure	#20

> Les besoins vagues du premier rapport (« temps acceptable », « plusieurs centaines d'utilisateurs ») ont été reformulés en seuils vérifiables (ENF-03 : p95 < 500 ms à 50 utilisateurs ; ENF-04 : contenu ≥ 80 % de la largeur à 360 px). Cette revue des exigences est la première technique statique appliquée.

5. Outil de gestion des tests et des exigences : GitHub (objectif 3)

5.1 Choix et organisation

L'équipe gère déjà son code sur GitHub. Nous y avons aussi géré les exigences, les cas de test, les campagnes et les anomalies, pour garder la traçabilité au même endroit que le code et les correctifs.

Fonction d'un outil de gestion de test	Réalisation dans GitHub
Référentiel des exigences	20 issues `[EF-xx]` / `[ENF-xx]`, libellés `exigence`, `fonctionnelle` / `non fonctionnelle`, criticité
Cas de test rattachés aux exigences	32 issues `[TU/TI/TS/TA/TNF/TR/TM-xx]` : niveau, type, technique, mode, fichier ; lien « Vérifie #n »
Campagnes d'exécution	Jalons « Itération 1 — build v1.0.0 » et « Itération 2 — build v1.0.1 »
Résultats d'exécution	Commentaire de résultat par itération sur chaque cas (x/y vérifications), fermeture une fois exécuté
Gestion des anomalies	18 issues `[ANO-xx]` : gravité, exigence, test détecteur, reproduction
Traçabilité anomalie → correctif → re-test	Pull Request #71 « Fixes #n » : la fusion ferme les anomalies corrigées
Suivi visuel (tableau de bord de la campagne)	GitHub Projects : colonnes du cycle de vie (À faire → En cours → Corrigée → Re-testée → Fermée), champ « Catégorie » (Exigence / Cas de test / Anomalie) — projet public
Rapports	Filtres par libellé et jalon ; matrice générée depuis le référentiel (`qa/referentiel.mjs`)

Pour éviter une saisie manuelle de 70 issues, le référentiel est écrit une seule fois (`qa/referentiel.mjs`) puis publié par l'API GitHub (`qa/github/publish.mjs`). Le même fichier génère les tableaux de ce rapport, ce qui garantit leur cohérence.

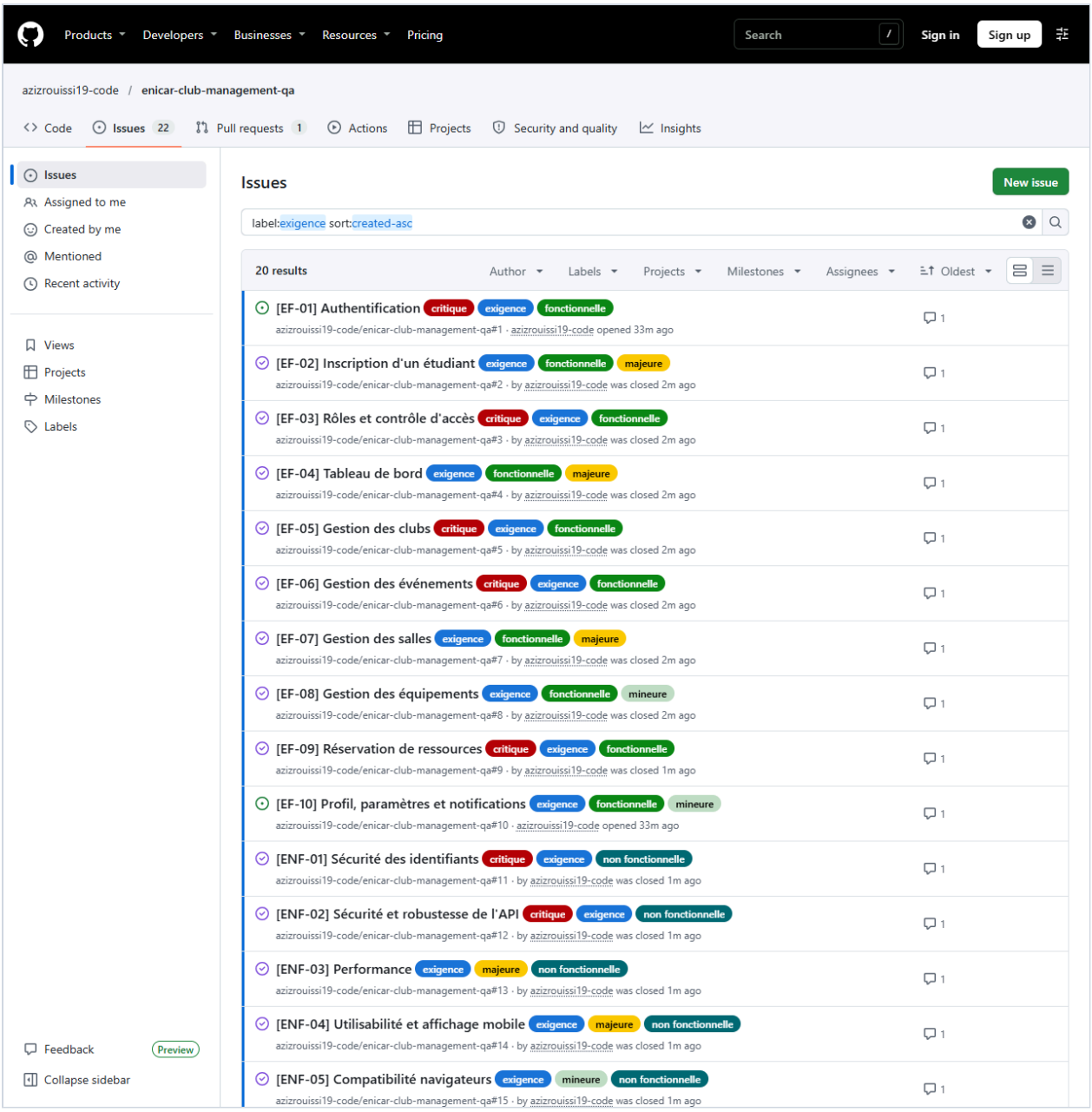


Figure 4 – Issues « exigence » du dépôt

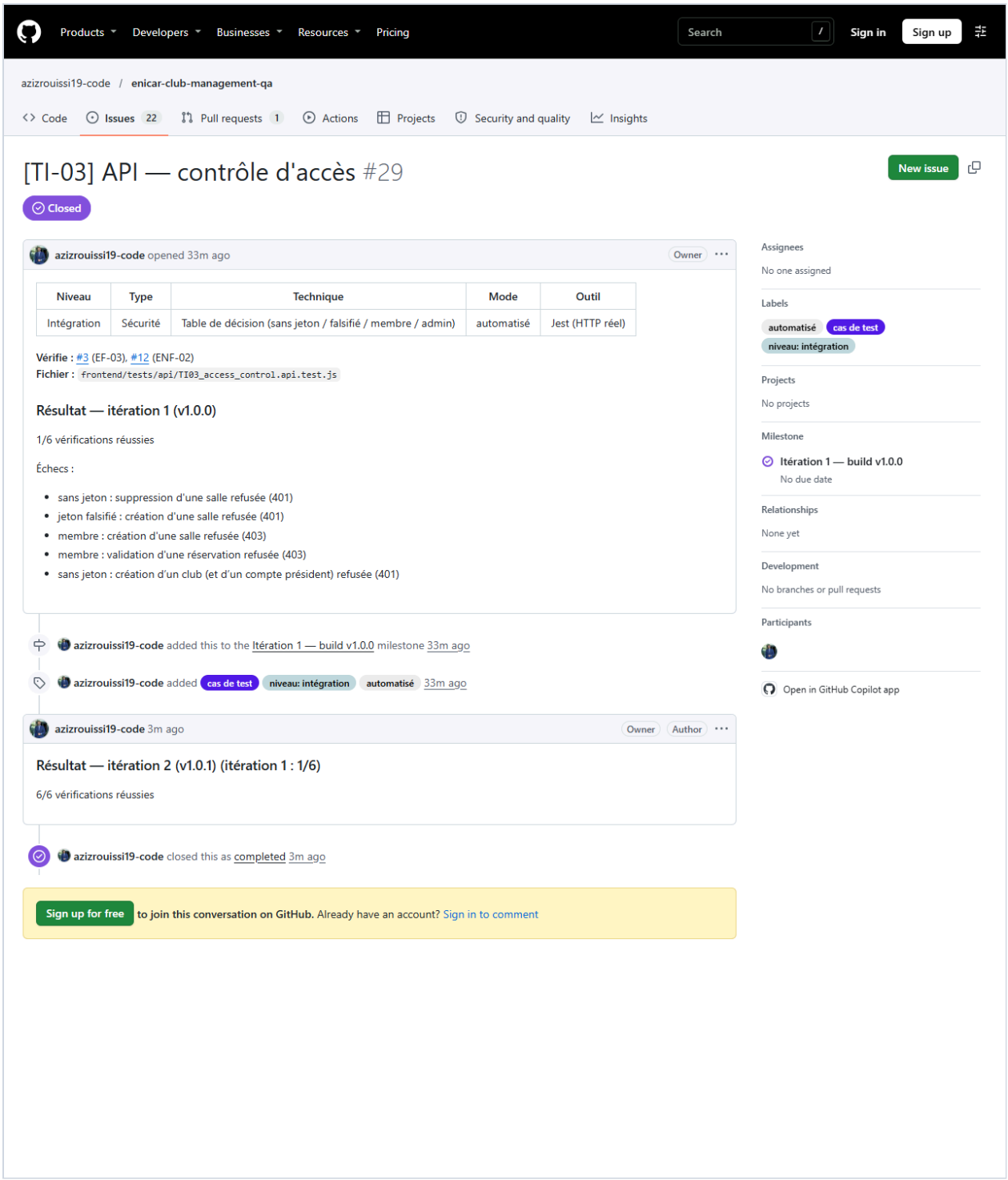


Figure 5 – Cas de test TI-03 : lien vers les exigences, résultats des deux itérations

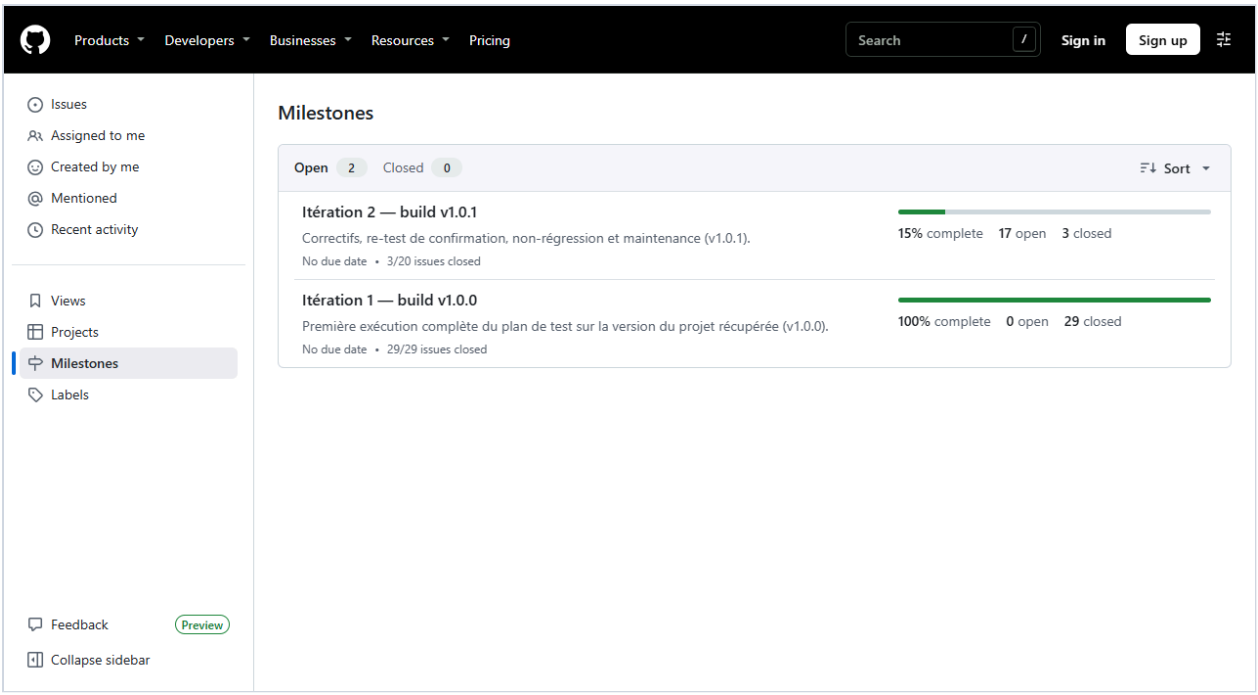


Figure 6 – Jalons = campagnes de test

Le tableau GitHub Projects regroupe les 70 issues selon le cycle de vie défini au § 6.9. À la fin de l'itération 2, les 4 éléments « À faire » sont l'exigence EF-10 et les anomalies ouvertes (ANO-12, 16, 17). Les 3 éléments « En cours » sont les exigences satisfaites avec réserve. Les colonnes « Corrigée » et « Re-testée » sont vides parce que toutes les anomalies corrigées ont été re-testées puis fermées.

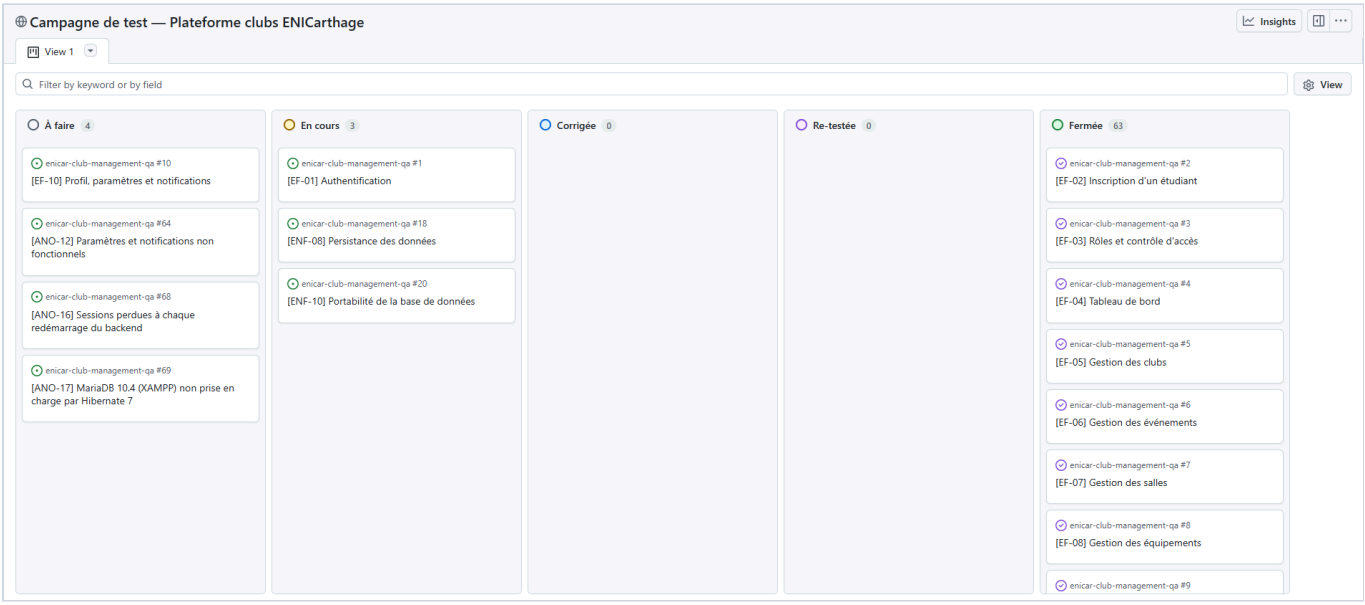


Figure 7 – Tableau GitHub Projects de la campagne (cycle de vie des anomalies)

5.2 Matrice de traçabilité exigences → cas de test

Exigence	Criticité	Cas de test qui la vérifient	Verdict (v1.0.1)
EF-01 Authentification	Critique	TU-01, TU-06, TI-01, TS-01, TR-01, TR-02	Satisfaite avec réserve (ANO-16)
EF-02 Inscription d'un étudiant	Majeure	TU-01, TU-06, TI-02, TS-02, TA-03, TR-01	Satisfaite
EF-03 Rôles et contrôle d'accès	Critique	TU-02, TU-04, TI-03, TS-01, TA-03, TR-01, TR-02	Satisfaite
EF-04 Tableau de bord	Majeure	TU-05, TS-06, TA-02, TR-01	Satisfaite

EF-05 Gestion des clubs	Critique	TU-03, TU-04, TI-04, TS-03, TA-01, TR-01, TR-02	Satisfaite
EF-06 Gestion des événements	Critique	TU-03, TU-04, TI-05, TS-04, TA-02, TR-01, TR-02	Satisfaite
EF-07 Gestion des salles	Majeure	TU-03, TI-07, TA-01, TR-01	Satisfaite
EF-08 Gestion des équipements	Mineure	TI-07	Satisfaite
EF-09 Réservation de ressources	Critique	TU-03, TU-04, TI-06, TS-05, TA-01, TA-03, TR-01, TR-02	Satisfaite
EF-10 Profil, paramètres et notifications	Mineure	TU-05, TU-06	Non satisfaite (ANO-12)
ENF-01 Sécurité des identifiants	Critique	TU-01, TI-01, TR-01	Satisfaite
ENF-02 Sécurité et robustesse de l'API	Critique	TI-01, TI-02, TI-03, TI-04, TI-06, TR-01	Satisfaite
ENF-03 Performance	Majeure	TNF-01, TR-02	Satisfaite
ENF-04 Utilisabilité et affichage mobile	Majeure	TU-03, TS-01, TS-07, TR-01	Satisfaite
ENF-05 Compatibilité navigateurs	Mineure	TS-08	Satisfaite
ENF-06 Maintenabilité	Mineure	TNF-03, TNF-04	Satisfaite
ENF-07 Sécurité des dépendances	Majeure	TNF-02, TR-01	Satisfaite
ENF-08 Persistance des données	Majeure	TNF-05, TM-01	Satisfaite avec réserve (ANO-16)
ENF-09 Configuration sécurisée	Majeure	TNF-03	Satisfaite
ENF-10 Portabilité de la base de données	Mineure	TM-01	Satisfaite avec réserve (ANO-17)

> **Couverture des exigences : 20/20 (100 %)** — chaque exigence est vérifiée par au moins un cas de test.

6. Plan de test (objectif 4)

Ce plan suit la structure de la norme IEEE 829 / ISO/IEC/IEEE 29119-3.

6.1 Objet et périmètre

Dans le périmètre	Hors périmètre
Authentification, inscription, rôles · clubs · événements · salles · équipements · réservations · tableau de bord · profil/paramètres/notifications · sécurité de l'API · performance · affichage mobile · compatibilité · maintenabilité · dépendances · persistance · portabilité de la base	Déploiement en production ; envoi d'e-mails (non implémenté) ; accessibilité avancée (lecteurs d'écran)

6.2 Stratégie : approche basée sur les risques

Risque produit	Prob.	Impact	Priorité	Tests prévus
Action d'administration par un utilisateur non autorisé	Élevée	Critique	P1	Revue de code, TI-03, TU-02, TU-04
Fuite des mots de passe	Moyenne	Critique	P1	Revue, TI-01, TU-01
Double réservation d'une salle	Moyenne	Majeur	P1	TI-06, TS-05, TA-03
Événement dépassant la capacité d'une salle	Moyenne	Majeur	P1	TI-05, TA-02
Données incohérentes affichées	Moyenne	Majeur	P2	TU-05, TS-06
Erreur 500 sur une saisie invalide	Moyenne	Moyen	P2	TI-01 à TI-07
Lenteur en période d'inscription	Faible	Majeur	P2	TNF-01
Mauvais affichage sur smartphone	Moyenne	Moyen	P2	TS-07, TA-03
Dépendances vulnérables	Élevée	Majeur	P2	TNF-02
Perte de données au redémarrage	Faible	Critique	P2	TNF-05

6.3 Niveaux de test (cycle en V)

Phase de développement	Niveau de test	Cas	Objectif
Besoins (EF / ENF)	Acceptation	TA-01 … TA-03	Valider le besoin du point de vue de chaque profil
Spécifications	Système	TS-01 … TS-08, TNF-01, TNF-05	Application complète dans un navigateur réel
Conception (API REST, modèle JPA)	Intégration	TI-01 … TI-07	API + sécurité + base de données réelles
Codage	Unitaire / composant	TU-01 … TU-06, TNF-03, TNF-04	Composants React et logique isolés (API simulée)

6.4 Types de test par niveau

Niveau \ Type	Fonctionnel	Non fonctionnel	Structurel	Lié aux changements	Maintenance
Unitaire	TU-01 … TU-06	TU-01 (sécurité), TNF-03	TNF-04 (Jest)	TR-01, TR-02	—
Intégration	TI-01, 02, 04 … 07	TI-03 (sécurité), TI-01	TNF-04 (JaCoCo)	TR-01, TR-02	TM-01
Système	TS-01 … TS-06	TS-07, TS-08, TNF-01, TNF-02, TNF-05	—	TR-02	TM-01
Acceptation	TA-01 … TA-03	TA-03 (mobile)	—	—	—

6.5 Techniques de test

Famille	Technique	Application

Statique	Revue des exigences	Reformulation des besoins en seuils mesurables (§ 4)
	Revue de code	Contrôleurs, sécurité, configuration : ANO-01, ANO-02 et ANO-15 trouvées avant exécution
	Analyse statique outillée	ESLint (JavaScript), PMD/CPD (Java)
	Analyse de composition	`npm audit` sur les dépendances de production
Dynamique boîte noire	Partitions d'équivalence	Identifiants valides/invalides, adresses institutionnelles ou non (TU-01, TI-01, TI-02)
	Valeurs limites	Mot de passe 5/6 caractères ; capacité 40/41 ; capacité 0/−10 (TU-06, TI-02, TI-05, TI-07)
	Table de décision	Routes protégées (6 règles), rôle × action (TU-02, TU-04, TI-03)
	Transitions d'états	Réservation PENDING → APPROVED / REJECTED (TU-04, TI-06)
	Cas d'utilisation	Scénarios multi-acteurs E2E et d'acceptation
Boîte blanche	Couverture instructions / branches	Jest --coverage, JaCoCo (TNF-04)
Expérience	Tests exploratoires	Revue visuelle des captures (ANO-09, ANO-11), appel direct de l'API (TA-03), redémarrage (TNF-05)

6.6 Tests manuels et automatisés

Mode	Cas	Outils
Automatisé	TU-01, TU-02, TU-03, TU-04, TU-05, TU-06, TI-01, TI-02, TI-03, TI-04, TI-05, TI-06, TI-07, TS-01, TS-02, TS-03, TS-04, TS-05, TS-06, TS-07, TS-08, TNF-01, TNF-02, TNF-04, TR-01, TR-02, TM-01	Jest, Playwright, k6, npm audit, JaCoCo
Manuel	TA-01, TA-02, TA-03, TNF-03, TNF-05	Navigateur, fiches d'acceptation, revue de code

6.7 Environnement et données de test

- **Unitaires :** jsdom, API simulée avec les mêmes données que le `DataLoader` ; stockage local vidé avant chaque test.
- **Intégration et système :** backend réel sur MySQL 8.4 ; chaque test crée ses propres données (suffixe unique) pour être rejouable.
- **Acceptation :** base réinitialisée (données initiales uniquement) ; comptes admin@enicar.tn et ahmed@enicar.tn.
- **Navigateurs :** Microsoft Edge et Firefox, 1440 × 900 et 360 × 800.

6.8 Critères d'entrée, de sortie et de suspension

Critère	Définition
Entrée	Application compilée et démarrée ; base accessible ; exigences et cas publiés sur GitHub
Sortie	100 % des cas exécutés ; aucune anomalie critique ou majeure ouverte ; 100 % des exigences couvertes ; couverture des instructions ≥ 70 % ; p95 < 500 ms
Suspension	Environnement indisponible ou anomalie bloquante empêchant plus de 30 % des tests
Reprise	Correctif livré ; re-test de confirmation puis non-régression complète

6.9 Gestion des anomalies

Cycle de vie : Détectée → Confirmée → Corrigée → Re-testée → Fermée. **Gravité :** Critique (sécurité, perte de données), Majeure (fonction dégradée), Mineure (gêne). Chaque anomalie est une issue `[ANO-xx]` liée à l'exigence et au test qui l'a détectée ; elle est fermée par la Pull Request qui la corrige.

6.10 Planning et livrables

Phase	Livrables
1. Installation	Environnement Docker, application v1.0.0 lancée
2. Analyse des besoins	Exigences EF / ENF (issues GitHub)
3. Conception des tests	Cas de test, matrice de traçabilité, plan de test
4. Itération 1 (v1.0.0)	Statiques, unitaires, intégration, système, non fonctionnels ; anomalies
5. Correctifs	Branche `iteration-2-correctifs`, Pull Request #71
6. Itération 2 (v1.0.1)	Re-test, non-régression, acceptation, persistance, maintenance
7. Clôture	Bilan, verdict, rapport

7. Exécution du plan de test (objectif 5)

7.1 Tests statiques

7.1.1 Revue de code

Avant toute exécution, les contrôleurs et la configuration ont été relus en les confrontant aux exigences. Trois défauts graves ont été trouvés, puis confirmés par les tests dynamiques :

```
// SecurityConfig.java — toutes les requêtes sont autorisées (EF-03, ENF-02)        -> ANO-01
.requestMatchers("/api/**").permitAll() // For development, allow all.
// AuthController.java — jeton prévisible, jamais vérifié ; l'entité User est renvoyée telle quelle
String dummyToken = "dummy-jwt-token-" + userOpt.get().getId();                      // -> ANO-01
response.put("user", userOpt.get());   // contient le hash BCrypt du mot de passe      -> ANO-02
// application.properties — secret et compte root sans mot de passe dans le dépôt     -> ANO-15
jwt.secret=94a08da1fe…   spring.datasource.password=   spring.jpa.show-sql=true
```

7.1.2 Analyse statique outillée

Outil	v1.0.0	v1.0.1
ESLint (frontend)	8 erreurs : 5 fonctions appelées dans `useEffect` avant leur déclaration, variables inutilisées, export mixte dans `AuthContext`	0 erreur
PMD 7.7 (backend)	2 alertes (classe utilitaire, `throws Exception`)	5 alertes : 7 alertes introduites par les correctifs (`toLowerCase` sans `Locale`, exception d'origine perdue…) ont été corrigées ; restent 2 complexités acceptées et 3 faux positifs liés au framework
CPD	0 duplication	0 duplication
npm audit (production)	5 hautes : axios, react-router(-dom), follow-redirects, form-data	0 après `npm audit fix`

> PMD ne trouve presque rien dans le backend (code généré par Lombok, peu de logique), alors que la revue de code trouve les deux anomalies critiques. Un outil d'analyse statique ne remplace pas la lecture du code par rapport aux exigences.

7.2 Tests unitaires et de composants (Jest + React Testing Library)

Les composants React sont testés isolément dans jsdom ; le module `api/axios` est remplacé par une API simulée. Exemple : la table de décision des routes protégées.

```
test.each([
  ["R1 non connecté, sans rôle exigé",     "PAGE LOGIN",     null,           undefined],
  ["R5 connecté, rôle non autorisé",       "PAGE DASHBOARD", USERS.member,   ["ADMINISTRATOR"]],
  ["R6 connecté, un des rôles autorisés",  "PAGE PROTEGEE",  USERS.president, ["ADMINISTRATOR",
"CLUB_PRESIDENT"]],
  /* … 6 règles */
])("%s -> %s", (_r, expected, user, roles) => {
  renderAt(user, roles);
  expect(screen.getByText(expected)).toBeInTheDocument();
});
```

Cas	Objet	Technique	Itération 1	Itération 2
TU-01	Contexte d'authentification	Partitions d'équivalence	8/9	9/9

TU-02	Routes protégées	Table de décision (6 règles)	6/6	6/6
TU-03	Validation des formulaires	Partitions, valeurs limites	10/12	12/12
TU-04	Interface selon le rôle	Table de décision rôle × action, transition d'états	17/19	19/19
TU-05	Tableau de bord et menus	Partitions	4/9	9/9
TU-06	Connexion, inscription, routage, pages de compte	Valeurs limites (mot de passe 5/6)	14/16	14/16
Total Jest « unit »			**59/71**	**69/71**

Échecs de l'itération 1 : hash du mot de passe conservé (ANO-02), tableau de bord et badges fictifs (ANO-08), membre pouvant gérer les événements et club codé en dur (ANO-10), échec silencieux de la création de club (ANO-06), paramètres et notifications non fonctionnels (ANO-12). Les 2 échecs restants en itération 2 concernent ANO-12, qui n'a pas été corrigée.

7.3 Tests d'intégration de l'API (Jest, requêtes HTTP réelles)

Le projet Jest « api » envoie de vraies requêtes au backend Spring Boot connecté à MySQL. Exemple : valeurs limites de la capacité d'une salle.

```
test.each([["participants > capacité de la salle (41 pour 40)", 41], ["participants négatifs", -5]])(
  "refus : %s -> 400", async (_c, attendees) => {
    const room = await newRoom(40);
    const r = await post("/events", { title: `Evt ${uid()}`, clubId: 1, date: futureDate(),
                                      room: room.name, attendees }, admin);
    expect(r.status).toBe(400);   // v1.0.0 : 200 -> ANO-04
});
```

Cas	Objet	Technique	Itération 1	Itération 2
TI-01	API — connexion	Partitions, robustesse	9/10	10/10
TI-02	API — inscription	Partitions, valeurs limites	4/9	9/9
TI-03	API — contrôle d'accès	Table de décision (sans jeton / falsifié / membre / admin)	1/6	6/6
TI-04	API — clubs	Partitions	2/4	4/4
TI-05	API — événements	Valeurs limites (capacité 40/41), partitions	2/7	7/7
TI-06	API — réservations	Transitions d'états, partitions	1/5	5/5
TI-07	API — salles et équipements	Valeurs limites (capacité 0, −10)	2/4	4/4
Total Jest « api »			**21/45**	**45/45**

> **Itération 1 :** l'API accepte toutes les requêtes sans jeton (TI-03 : 5 échecs sur 6), renvoie le hash du mot de passe (TI-01), n'applique aucune règle métier (TI-02, TI-05, TI-06, TI-07) et répond par une erreur 500 à une réservation inexistante ou à un e-mail de président déjà utilisé.

7.4 Tests structurels (couverture de code)

Mesure	Itération 1	Itération 2	Objectif
Frontend — instructions (Jest)	80,1 %	79,0 %	≥ 70 %
Backend — instructions (JaCoCo)	98 %	75 %	≥ 70 %
Backend — branches (JaCoCo)	88 %	75 %	—

> **Analyse :** à l'itération 1, le backend est couvert à 98 % alors que 24 tests d'API échouent. Une règle qui n'existe pas dans le code (contrôle d'accès, capacité, double réservation) ne peut pas être mesurée par la couverture. À l'itération 2, la couverture baisse à 75 % parce que les correctifs ajoutent des branches d'erreur et que le `DataLoader` ne s'exécute plus (base déjà remplie). Les branches non couvertes, visibles en jaune et rouge dans le rapport, indiquent les prochains tests à écrire.

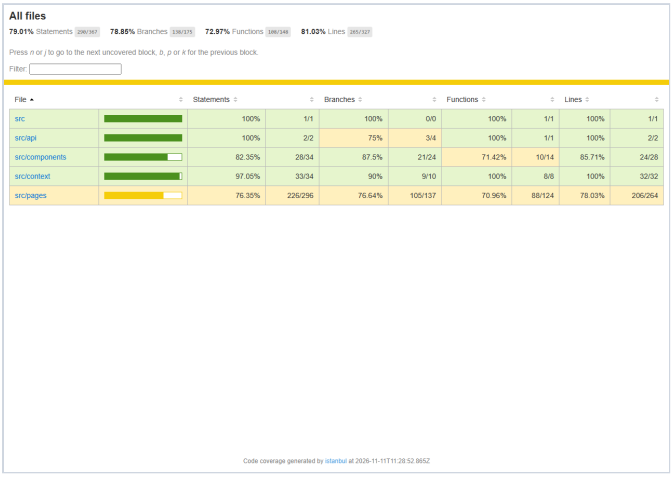


Figure 8 – Couverture du frontend (Jest)

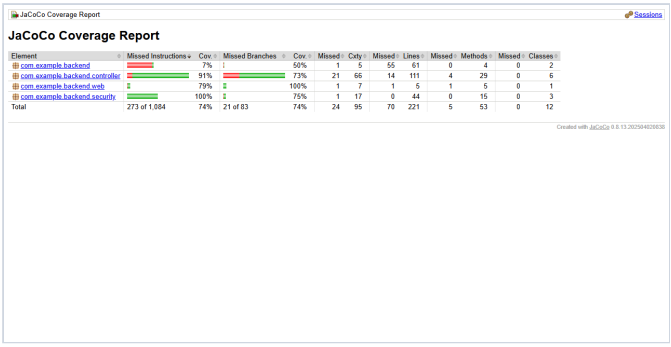


Figure 9 – Couverture du backend (JaCoCo)

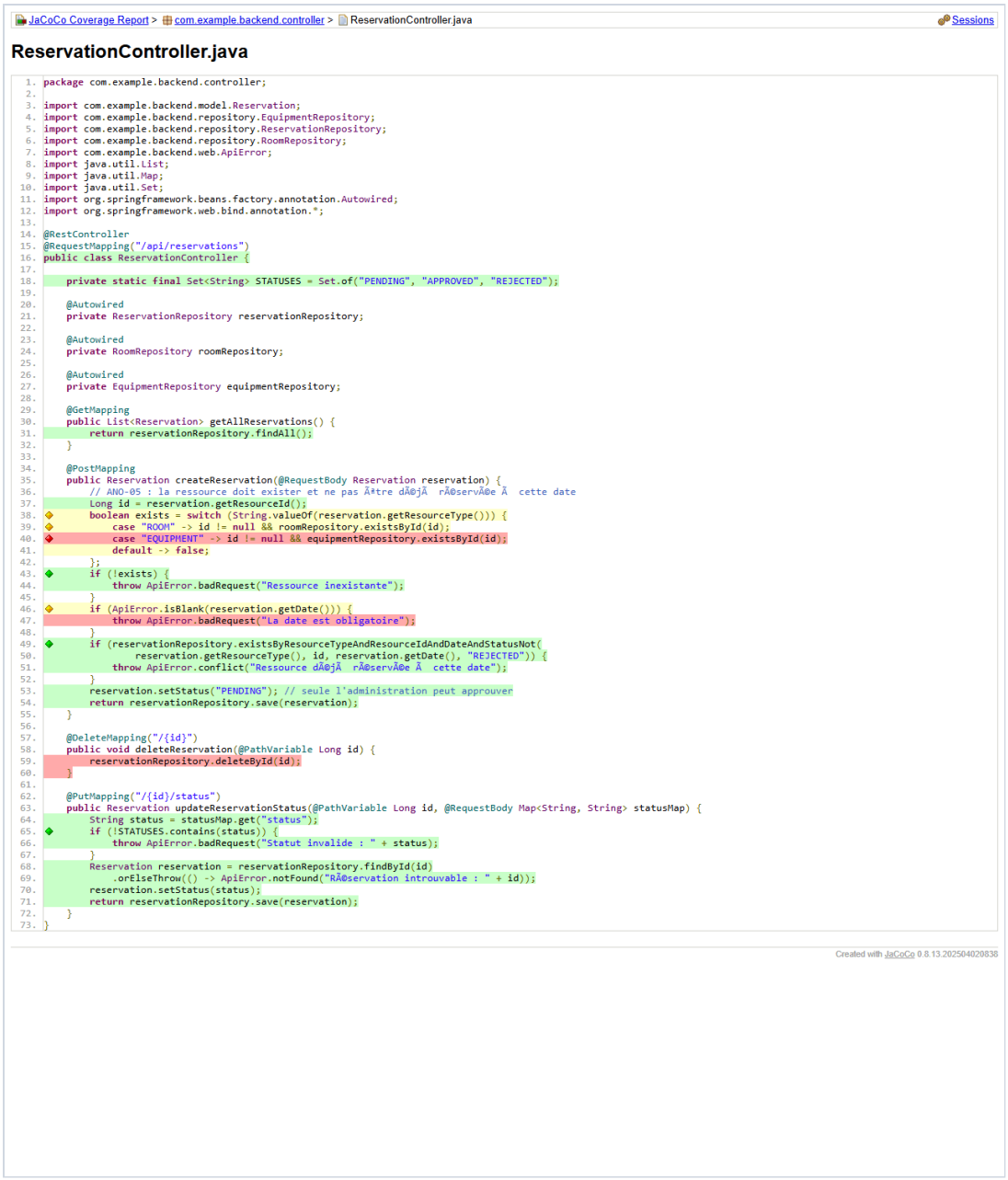


Figure 10 – Vue source JaCoCo : branches couvertes (vert), partielles (jaune), non couvertes (rouge)

7.5 Tests système (E2E avec Playwright)

Chaque scénario traverse toute l'application (React → API → MySQL) dans un vrai navigateur, sous Edge et sous Firefox (TS-08, compatibilité).

Cas	Scénario	Itération 1	Itération 2
TS-01	Authentification de bout en bout	4/6	6/6
TS-02	Inscription puis reconnexion	2/2	2/2
TS-03	Création d'un club et de son président	2/2	2/2
TS-04	Création d'un événement par le président	2/2	2/2
TS-05	Réservation par un membre, validation par l'administrateur	2/2	2/2
TS-06	Tableau de bord cohérent avec la base	0/2	2/2
TS-07	Affichage smartphone 360 × 800	2/6	6/6
TS-08	Compatibilité Edge / Firefox	14/22	22/22
Total (11 scénarios × 2 navigateurs)		**14/22**	**22/22**

Deux anomalies vues uniquement grâce aux captures (test exploratoire) : (1) avec un mauvais mot de passe, la page se recharge et le message disparaît (ANO-09). Le test unitaire TU-06 ne l'avait pas détecté, car l'API y est simulée : c'est un défaut d'intégration. (2) Sur smartphone, il n'y a pas de défilement horizontal (le critère automatique passait), mais la barre latérale occupe 60 % de l'écran (ANO-11). Le critère du test a donc été renforcé : la zone de contenu doit occuper au moins 80 % de la largeur.

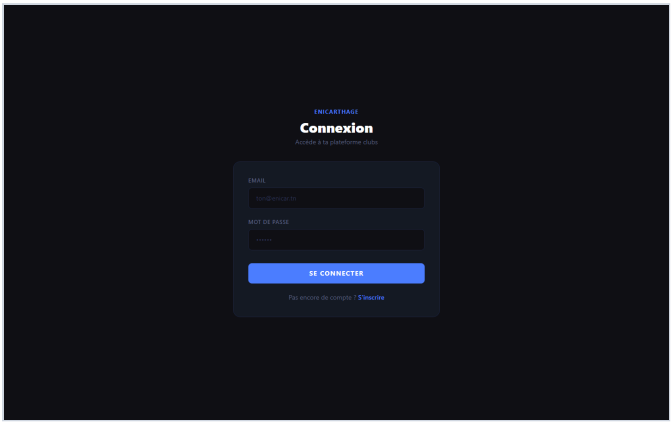


Figure 11 – v1.0.0 : mauvais mot de passe, aucun message (ANO-09)

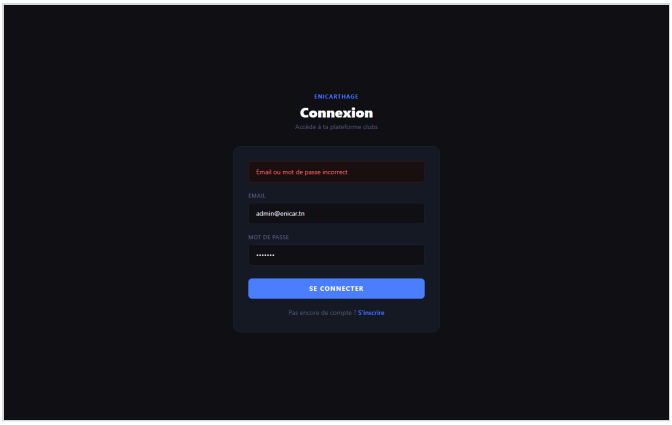


Figure 12 – v1.0.1 : message affiché

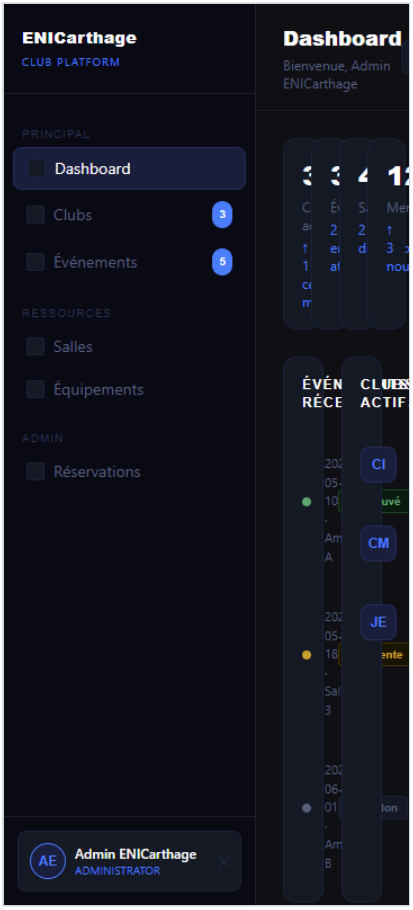


Figure 13 – v1.0.0 : smartphone 360 px (ANO-11)

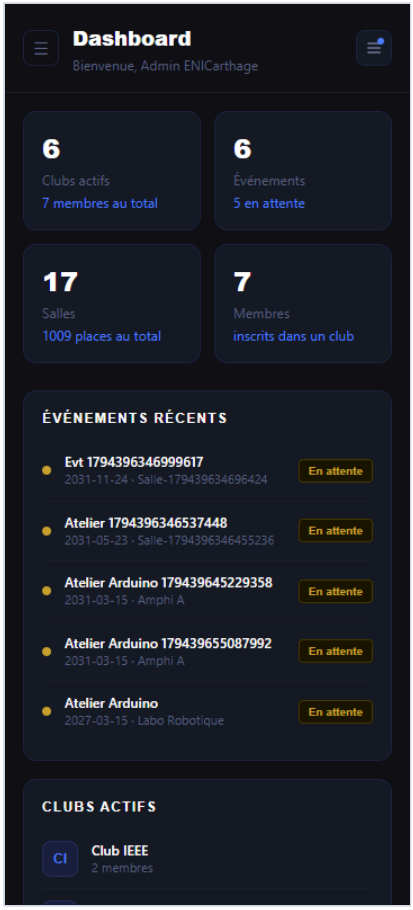


Figure 14 – v1.0.1 : menu en tiroir, contenu lisible

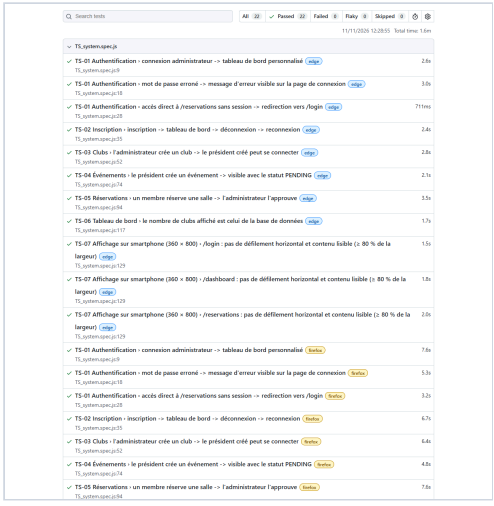


Figure 15 – Rapport HTML Playwright (v1.0.1)

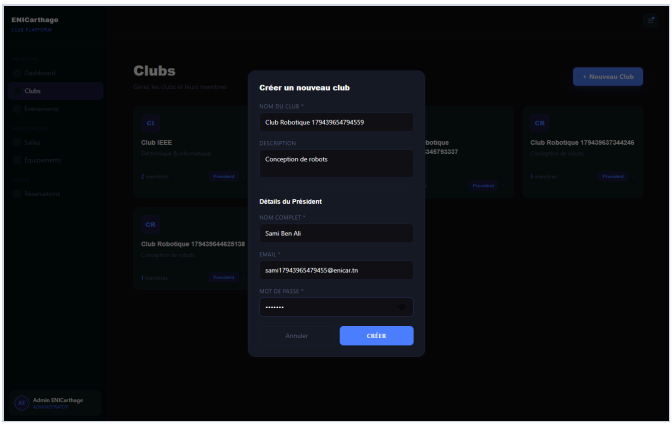


Figure 16 – TS-03 : l'administrateur crée un club et son président

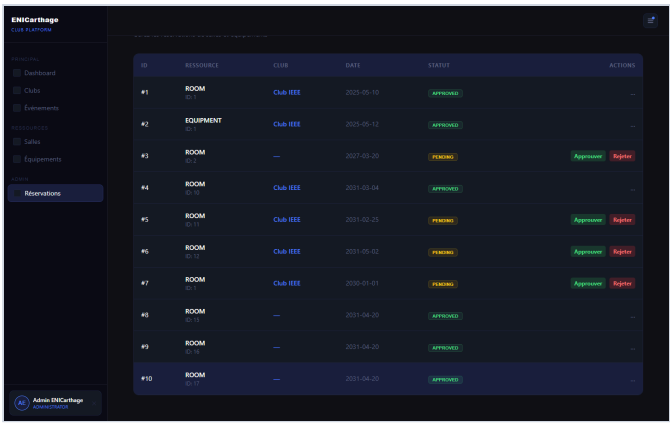


Figure 17 – TS-05 : réservation d'un membre approuvée par l'administrateur

7.6 Tests d'acceptation (manuels)

Les fiches TA-01 à TA-03 (`qa/acceptance/TA-fiches.md`) décrivent un parcours métier par profil (action → résultat attendu). Elles ont été exécutées pas à pas dans le navigateur sur la version v1.0.1, avec une base réinitialisée. Un script d'aide (`tests/acceptance/TA_acceptance.spec.js`) réalise les actions et prend une capture à chaque étape comme preuve.

TA-01 — Parcours administrateur

#	Action	Résultat attendu	Résultat obtenu
1	Se connecter avec admin@enicar.tn / admin123	Tableau de bord « Bienvenue, Admin ENICarthage » ; 1 club, 1 événement, 1 salle	Conforme
2	Salles → + Nouvelle Salle « Labo Robotique », capacité 0	Message d'erreur : capacité strictement positive	Conforme
3	Recommencer avec une capacité de 24	La salle « Labo Robotique — 24 places » apparaît	Conforme
4	Clubs → + Nouveau Club « Club Robotique », président Sami Ben Ali (sami@enicar.tn)	Le club apparaît dans la liste	Conforme
5	Créer « Club Théâtre » avec un président déjà inscrit (ahmed@enicar.tn)	Message : un compte existe déjà ; aucun club créé	Conforme
6	Réservations → approuver la réservation #2 (PENDING)	Le statut passe à APPROVED	Conforme

TA-02 — Parcours président de club

#	Action	Résultat attendu	Résultat obtenu
1	Se connecter avec sami@enicar.tn / secret1 (compte créé en TA-01)	Rôle CLUB_PRESIDENT affiché	Conforme
2	Événements → + Nouvel Événement « Atelier Arduino », Labo Robotique, 15/03/2027, 20 participants	Seul « Club Robotique » est proposé ; événement créé, statut PENDING, « Par Club Robotique »	Conforme
3	Créer « Compétition » dans Labo Robotique avec 25 participants (capacité 24)	Message : participants entre 0 et 24	Conforme
4	Observer l'événement « Hackathon IEEE 2025 » d'un autre club	Aucun bouton de suppression	Conforme
5	Ouvrir le tableau de bord	Événements = 2 dont « 1 en attente »	Conforme

TA-03 — Parcours membre de club

#	Action	Résultat attendu	Résultat obtenu

1	S'inscrire avec youssef@gmail.com	Refus : adresse institutionnelle @enicar.tn obligatoire	Conforme
2	S'inscrire avec youssef@enicar.tn / secret1	Tableau de bord, rôle CLUB_MEMBER	Conforme
3	Ouvrir Clubs puis Événements	Aucun bouton de création ni de suppression	Conforme
4	Réservations → réserver la salle n°2 (Labo Robotique) le 20/03/2027	Réservation créée avec le statut PENDING	Conforme
5	Réserver à nouveau la même salle à la même date	Message : ressource déjà réservée à cette date	Conforme
6	Test exploratoire : appeler directement l'API pour approuver sa propre réservation	Refus HTTP 403	Conforme
7	Smartphone (360 px) : ouvrir le menu ☰	Le menu s'ouvre en tiroir, le contenu reste lisible	Conforme

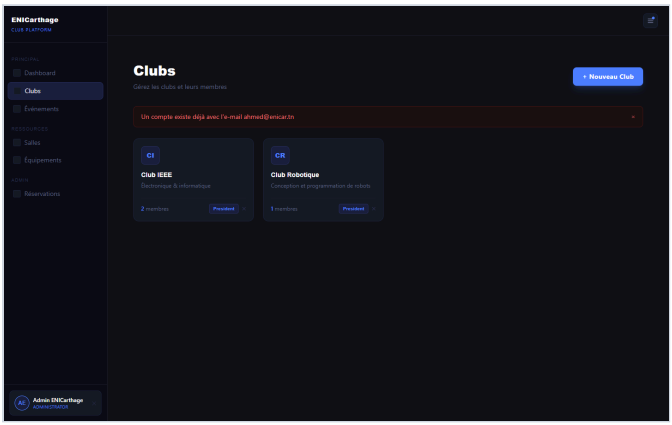


Figure 18 – TA-01 étape 5 : président déjà inscrit, message affiché

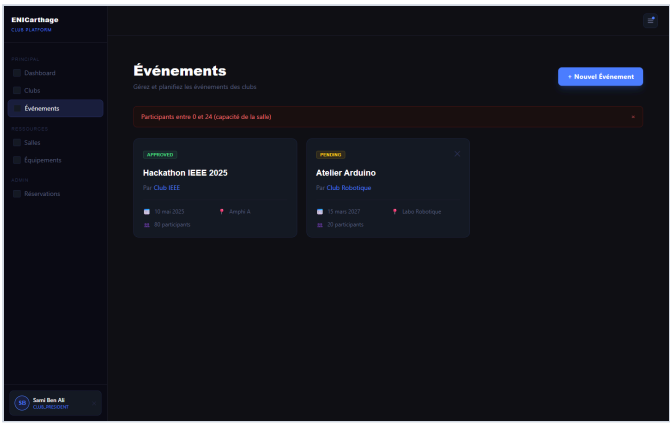


Figure 19 – TA-02 étape 3 : capacité de la salle dépassée

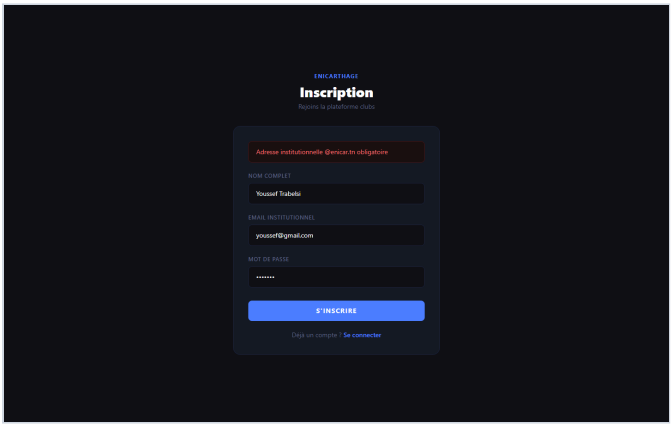


Figure 20 – TA-03 étape 1 : adresse non institutionnelle refusée

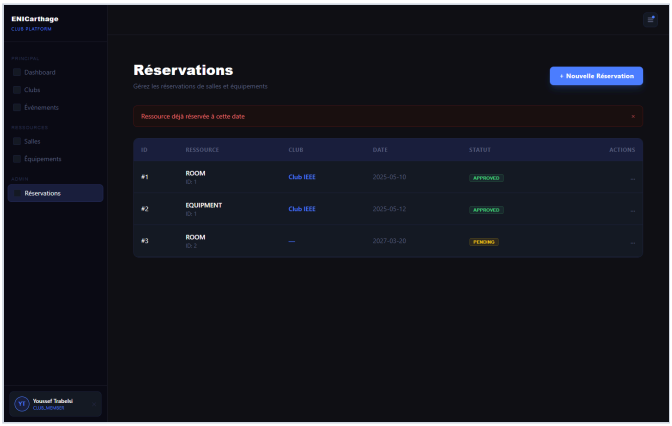


Figure 21 – TA-03 étape 5 : double réservation refusée

7.7 Tests non fonctionnels

Performance et fiabilité — test de charge k6 (TNF-01)

Profil : montée à 50 utilisateurs virtuels en 15 s, palier de 45 s, descente en 10 s. Chaque utilisateur se connecte puis consulte les cinq ressources. Seuils : p95 < 500 ms (ENF-03) et taux d'erreur < 1 %.

Mesure	Itération 1 (v1.0.0)	Itération 2 (v1.0.1, API sécurisée)
Requêtes	10 932 (155 req/s)	10 728 (153 req/s)
Temps moyen / médian	99,6 ms / 50,6 ms	104,9 ms / 68,3 ms
p95 (seuil 500 ms)	359 ms	357 ms

Taux d'erreur (seuil 1 %)	0 %	0 %

La vérification du jeton à chaque requête (une lecture en base) augmente légèrement le temps médian sans dégrader le p95. Remarque : les listes ne sont pas paginées ; leur temps de réponse augmentera avec le volume de données.

Sécurité

Contrôle	v1.0.0	v1.0.1
Suppression d'une salle sans jeton (TI-03)	200 accepté	401
Jeton falsifié « dummy-jwt-token-1 »	200 accepté	401
Membre qui crée une salle / approuve une réservation	200 accepté	403
Hash du mot de passe dans la réponse de connexion	présent	absent
Rôle envoyé par le client à l'inscription	ignoré	ignoré
Dépendances vulnérables en production	5 hautes	0

Persistance (TNF-05)

Le décompte des données est identique avant et après un redémarrage du backend (12 clubs, 17 événements, 35 salles, 4 équipements, 19 réservations). En revanche, les jetons de session sont conservés en mémoire : après un redémarrage, le jeton obtenu avant est refusé (401) et tous les utilisateurs doivent se reconnecter. C'est un effet de bord du correctif d'ANO-01 (ANO-16).

7.8 Tests liés aux changements et tests de maintenance

Correctifs appliqués (v1.0.1)

Anomalie	Correctif
ANO-01	`TokenService` (jeton aléatoire de 256 bits) + `TokenAuthenticationFilter` ; règles par rôle dans `SecurityConfig` ; réponses 401/403
ANO-02	`@JsonProperty(access = WRITE_ONLY)` sur le mot de passe ; le frontend retire aussi ce champ par précaution
ANO-03 à 07	Validations métier dans les contrôleurs ; erreurs 400 / 404 / 409 explicites (classe `ApiError`)
ANO-08	Tableau de bord calculé à partir de l'API ; suppression de `mock/data.js` et des badges codés en dur
ANO-09	L'intercepteur axios ne redirige plus sur un 401 venant de `/auth/*`
ANO-10	Événements gérés par le président et l'administrateur ; club du président sélectionné ; club de l'utilisateur pour les réservations
ANO-11	Barre latérale en tiroir sous 768 px (bouton ☰), grilles responsives
ANO-13, 14, 15	`npm audit fix` ; ESLint corrigé (hook `useAuth` séparé) ; configuration par variables d'environnement, CORS limité, `show-sql` désactivé

```
// SecurityConfig.java (v1.0.1) — ANO-01 : règles d'accès par rôle
.requestMatchers("/api/auth/**", "/error").permitAll()
.requestMatchers(HttpMethod.GET, "/api/clubs/**", "/api/events/**", "/api/rooms/**",
"/api/equipment/**").permitAll()
.requestMatchers("/api/clubs/**", "/api/rooms/**", "/api/equipment/**").hasRole(ADMIN)
.requestMatchers(HttpMethod.PUT, "/api/reservations/*/status").hasRole(ADMIN)
.requestMatchers("/api/events/**").hasAnyRole(ADMIN, PRESIDENT)
.anyRequest().authenticated()
```

Re-test de confirmation (TR-01) et non-régression (TR-02)

Le re-test ré-exécute les cas qui avaient échoué. La non-régression ré-exécute toutes les suites : Jest unit 69/71, Jest API 45/45, E2E 22/22, et k6 reste conforme.

> **Régression détectée puis corrigée (ANO-18) :** la première non-régression E2E a échoué une fois sous Edge (TS-04). Le correctif d'ANO-10 calcule le club du président à partir de la liste des clubs ; si le président clique sur « + Nouvel Événement » avant la fin du chargement, la page affiche « Aucun club associé à votre compte ». Correctif : bouton désactivé tant que les clubs ne sont pas chargés. Re-test : TS-04 répété 10 fois, 10/10 réussis. Ce cas montre l'intérêt de la non-régression : un correctif peut introduire un nouveau défaut.

Pour rendre ce défaut intermittent reproductible, un test de régression dédié (`tests/regression/ANO18.spec.js`) retarde volontairement de 2 s la réponse de `/api/clubs`. Il échoue sur le code d'avant le correctif et réussit après : il fait désormais partie de la suite de non-régression.

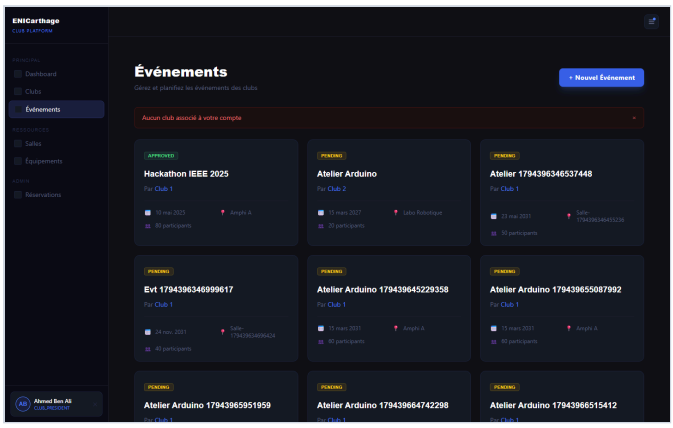


Figure 22 – ANO-18 reproduite (code avant correctif) : « Aucun club associé »

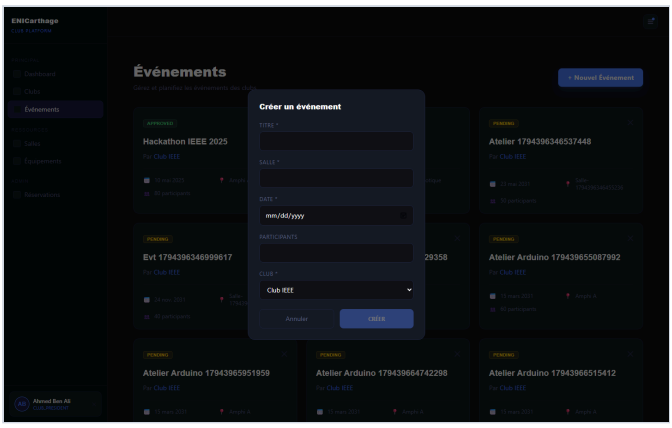


Figure 23 – Après correctif : bouton désactivé pendant le chargement, puis création possible

Tests de maintenance (TM-01)

La configuration du projet vise XAMPP, qui embarque MariaDB 10.4. Le backend v1.0.1 a été porté sur MariaDB 10.4 (Docker) sans modifier le code : **45/45 tests d'API réussis**. Hibernate 7 signale toutefois que cette version n'est plus prise en charge (avertissement HHH000511 ; MariaDB 10.4 est en fin de vie depuis 2024) : ANO-17. Recommandation : MySQL 8 ou MariaDB ≥ 10.6.

7.9 Suivi de la campagne sur GitHub

Les correctifs sont regroupés dans la branche `iteration-2-correctifs` et la Pull Request #71. Chaque ligne « Fixes #n » de la description ferme l'anomalie correspondante lors de la fusion, ce qui relie anomalie, correctif et résultats de re-test.

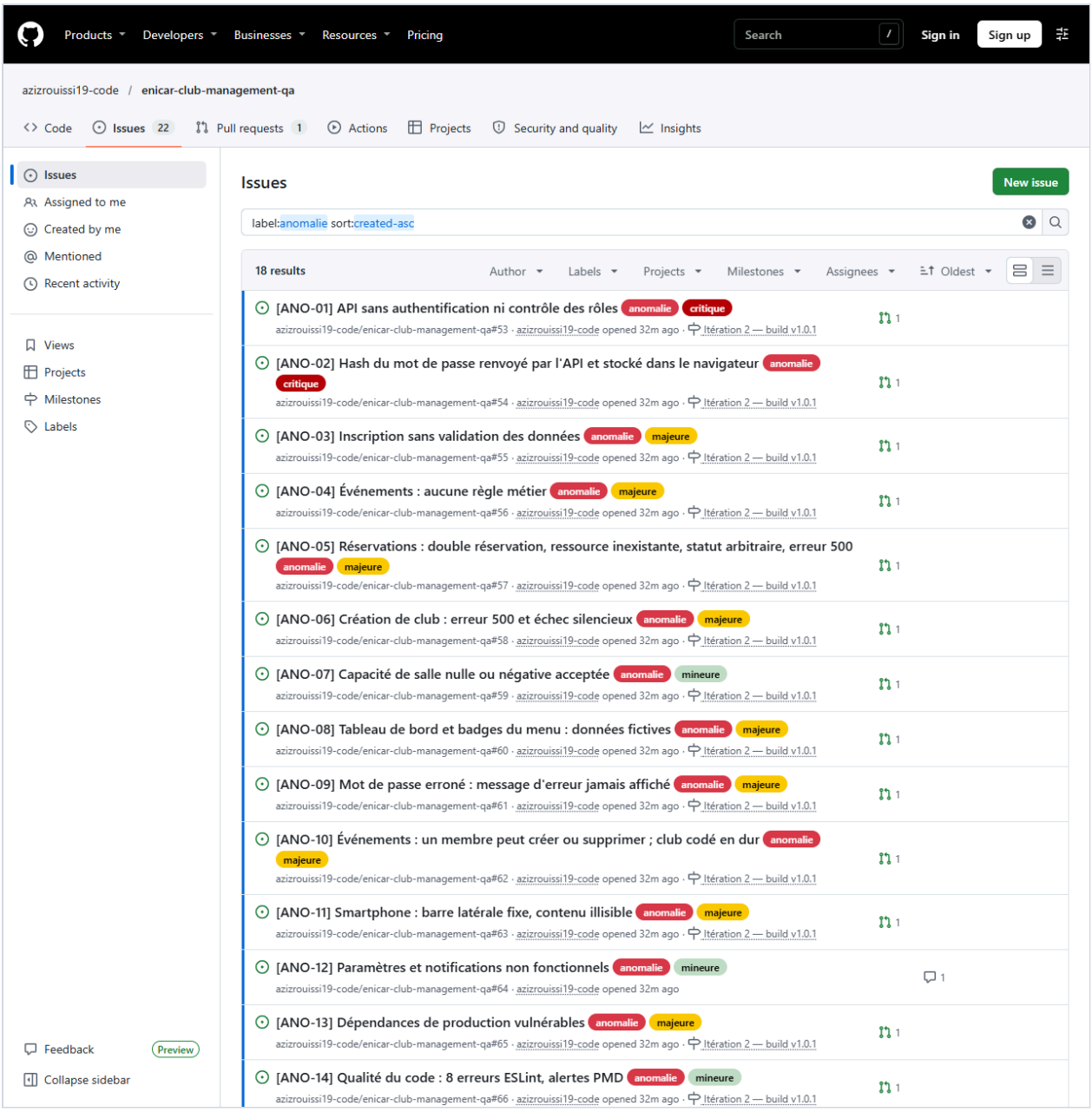


Figure 24 – Issues « anomalie » liées à la Pull Request de correction

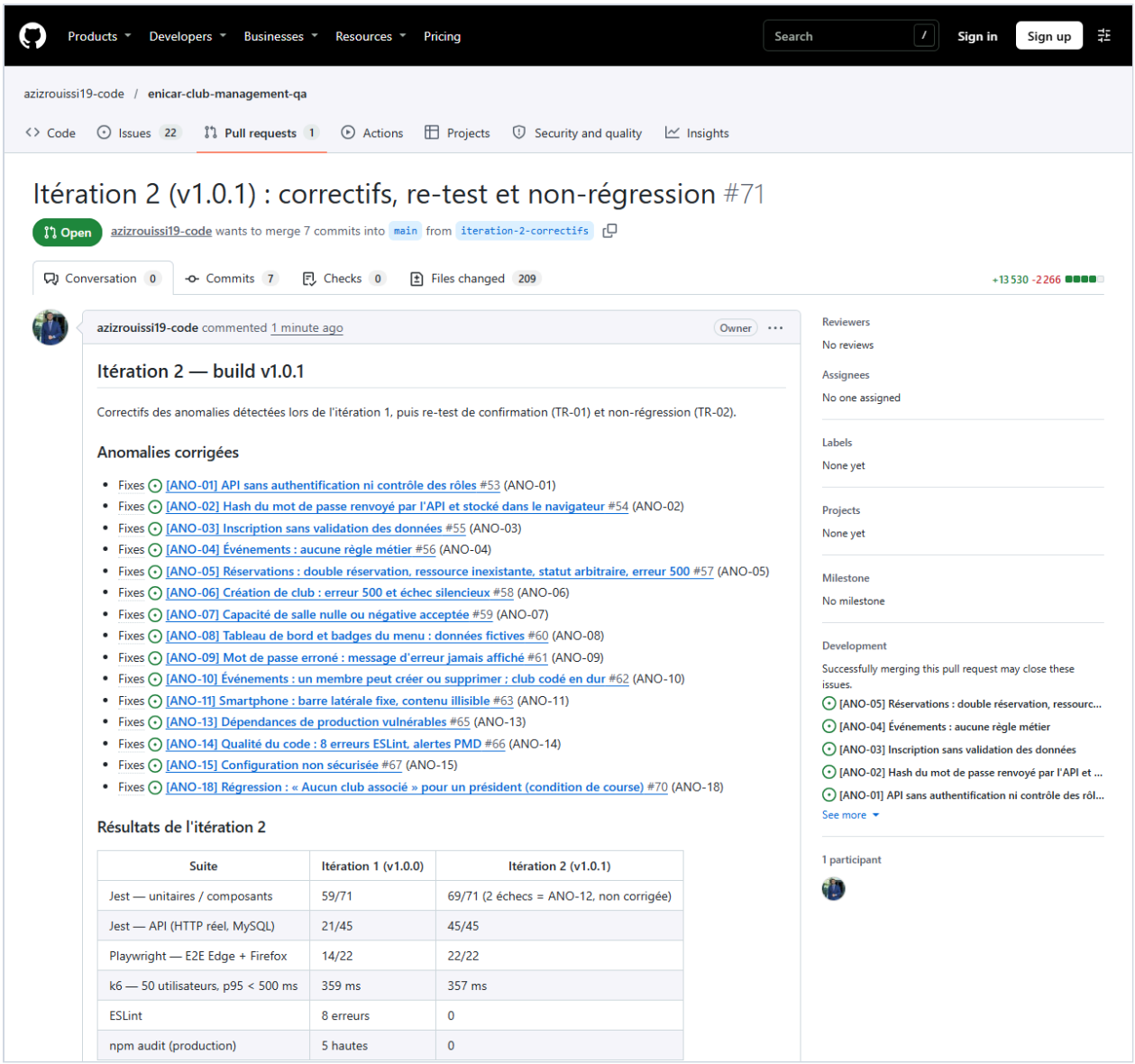


Figure 25 – Pull Request #71 : correctifs et résultats de l'itération 2

8. Bilan : anomalies, métriques et verdict de recette

8.1 Anomalies détectées

Réf.	Gravité	Description	Détectée par	Exigence	Statut
ANO-01 #53	Critique	**API sans authentification ni contrôle des rôles**	Revue de code, TI-03	EF-03, ENF-02	Corrigée et re-testée (v1.0.1)
ANO-02 #54	Critique	**Hash du mot de passe renvoyé par l'API et stocké dans le navigateur**	TI-01, TU-01	ENF-01	Corrigée et re-testée (v1.0.1)
ANO-03 #55	Majeure	**Inscription sans validation des données**	TI-02	EF-02	Corrigée et re-testée (v1.0.1)
ANO-04 #56	Majeure	**Événements : aucune règle métier**	TI-05	EF-06	Corrigée et re-testée (v1.0.1)
ANO-05 #57	Majeure	**Réservations : double réservation, ressource inexistante, statut arbitraire, erreur 500**	TI-06	EF-09, ENF-02	Corrigée et re-testée (v1.0.1)
ANO-06 #58	Majeure	**Création de club : erreur 500 et échec silencieux**	TI-04, TU-03	EF-05, ENF-04	Corrigée et re-testée (v1.0.1)
ANO-07 #59	Mineure	**Capacité de salle nulle ou négative acceptée**	TI-07	EF-07	Corrigée et re-testée (v1.0.1)
ANO-08 #60	Majeure	**Tableau de bord et badges du menu : données fictives**	TU-05, TS-06	EF-04	Corrigée et re-testée (v1.0.1)
ANO-09 #61	Majeure	**Mot de passe erroné : message d'erreur jamais affiché**	TS-01	EF-01, ENF-04	Corrigée et re-testée (v1.0.1)
ANO-10 #62	Majeure	**Événements : un membre peut créer ou supprimer ; club codé en dur**	TU-04, TU-03	EF-03, EF-06	Corrigée et re-testée (v1.0.1)
ANO-11 #63	Majeure	**Smartphone : barre latérale fixe, contenu illisible**	TS-07	ENF-04	Corrigée et re-testée (v1.0.1)
ANO-12 #64	Mineure	**Paramètres et notifications non fonctionnels**	TU-06	EF-10	Ouverte — fonctionnalité à développer
ANO-13 #65	Majeure	**Dépendances de production vulnérables**	TNF-02	ENF-07	Corrigée et re-testée (v1.0.1)
ANO-14 #66	Mineure	**Qualité du code : 8 erreurs ESLint, alertes PMD**	TNF-03	ENF-06	Corrigée et re-testée (v1.0.1)
ANO-15 #67	Majeure	**Configuration non sécurisée**	Revue de code, TNF-03	ENF-09	Corrigée et re-testée (v1.0.1)
ANO-16 #68	Mineure	**Sessions perdues à chaque redémarrage du backend**	TNF-05	ENF-08, EF-01	Ouverte — détectée en itération 2
ANO-17 #69	Mineure	**MariaDB 10.4 (XAMPP) non prise en charge par Hibernate 7**	TM-01	ENF-10	Ouverte — détectée en itération 2
ANO-18 #70	Majeure	**Régression : « Aucun club associé » pour un président (condition de course)**	TR-02, TS-04	EF-06	Corrigée et re-testée (v1.0.1)

8.2 Métriques

Indicateur	Itération 1 (v1.0.0)	Itération 2 (v1.0.1)
Tests Jest unitaires / composants	59/71	69/71

Tests Jest d'API (intégration)	21/45	45/45
Tests E2E Playwright (Edge + Firefox)	14/22	22/22
Étapes d'acceptation conformes	— (planifiées en itération 2)	18/18
Cas de test exécutés	24/32 (les cas TA, TNF-05, TR, TM appartiennent à l'itération 2)	32/32
Anomalies	15 détectées (2 critiques, 10 majeures, 3 mineures)	+3 détectées (ANO-16, 17, 18) ; 15 corrigées et re-testées ; 3 ouvertes (mineures)
Efficacité des techniques statiques	5 anomalies sur 18 trouvées sans exécuter le code (ANO-01, ANO-02, ANO-13, ANO-14, ANO-15 : revue de code, ESLint, npm audit), dont les 2 critiques	
Couverture des instructions (front / back)	80 % / 98 %	79 % / 75 %
p95 sous 50 utilisateurs	359 ms	357 ms

8.3 Évaluation des critères de sortie

Critère	Résultat	Verdict
100 % des cas planifiés exécutés	32/32	Atteint
Aucune anomalie critique ou majeure ouverte	Toutes corrigées et re-testées ; restent 3 mineures (ANO-12, 16, 17)	Atteint
100 % des exigences couvertes	20/20 ; 16 satisfaites, 3 avec réserve, 1 non satisfaite (EF-10)	Atteint
Couverture des instructions ≥ 70 %	Frontend 79 %, backend 75 %	Atteint
p95 < 500 ms à 50 utilisateurs	357 ms, 0 % d'erreurs	Atteint

Verdict : recette de la version v1.0.1 prononcée AVEC RÉSERVES. Tous les critères de sortie sont atteints, mais l'exigence EF-10 (paramètres et notifications) n'est pas satisfaite (ANO-12) et deux réserves mineures subsistent : sessions perdues au redémarrage (ANO-16) et MariaDB 10.4 non prise en charge (ANO-17). La version v1.0.0 aurait été refusée : 2 anomalies critiques et 10 majeures.

8.4 Recommandations

Sujet	Recommandation
ANO-16	Remplacer les jetons en mémoire par des JWT signés (clé fournie par une variable d'environnement) avec expiration
ANO-17	Utiliser MySQL 8 ou MariaDB ≥ 10.6 avec le pilote MariaDB
ANO-12	Développer les notifications (table et API) et l'action « Désactiver mon compte », ou retirer ces écrans de la version livrée
Données	Rattacher les membres à leurs clubs (adhésion), paginer les listes, choisir la salle dans une liste plutôt que la saisir
Intégration continue	Exécuter `npm test`, ESLint, `npm audit` et Playwright à chaque Pull Request (GitHub Actions) ; bloquer la fusion en cas d'échec

9. Conclusion

Cet atelier nous a permis de mener une démarche de test complète sur notre propre plateforme, de l'expression des besoins jusqu'au verdict de recette. GitHub a servi de référentiel unique : les exigences, les cas de test, les campagnes et les anomalies y sont des issues liées entre elles, et la Pull Request de correction relie chaque anomalie à son correctif et à son re-test.

La combinaison des techniques s'est révélée nécessaire. La revue de code a trouvé les défauts les plus graves avant toute exécution, alors que l'analyse outillée et la couverture (98 %) ne les voyaient pas. Les tests dynamiques avec Jest (partitions, valeurs limites, tables de décision, transitions d'états) les ont confirmés et mesurés. Les tests système dans deux navigateurs ont révélé un défaut d'intégration invisible au niveau unitaire (ANO-09). La revue visuelle des captures a montré la limite d'un critère automatique trop faible (ANO-11). Enfin, la non-régression a détecté un défaut introduit par un correctif (ANO-18), ce qui confirme que le cycle correction → re-test → non-régression est indispensable.

10. Annexe : rejouer les tests

```
# 1. Base de données et backend (avec couverture JaCoCo)
docker run -d --name enicar-mysql -e MYSQL_ALLOW_EMPTY_PASSWORD=yes -p 3306:3306 mysql:8.4
cd backend && ./mvnw -DskipTests package
java -javaagent:qa/tools/org.jacoco.agent-0.8.13-runtime.jar=output=tcpserver,port=6300 -jar
backend/target/backend-0.0.1-SNAPSHOT.jar

# 2. Tests (dossier frontend)
npm install
npm run test:unit            # Jest unitaires/composants + couverture
npm run test:api             # Jest intégration : requêtes HTTP réelles vers l'API
npx playwright test          # E2E Edge + Firefox (démarre Vite automatiquement)
npx playwright test -c playwright.acceptance.config.js   # aide à l'exécution des tests d'acceptation
npx eslint src && npm audit --omit=dev                     # analyse statique et dépendances

# 3. Backend : analyse statique, charge, couverture
cd backend && ./mvnw -Pqa pmd:pmd pmd:cpd
docker run --rm -v ./qa/perf:/scripts grafana/k6 run /scripts/charge-api.js
java -jar qa/tools/org.jacoco.cli-0.8.13-nodeps.jar dump --address localhost --port 6300 --destfile jacoco.exec

# 4. Référentiel GitHub
node qa/github/publish.mjs <owner/repo>       # exigences, cas, anomalies -> issues
node qa/github/iteration2.mjs <owner/repo>    # résultats de l'itération 2
node qa/report/build.mjs                      # génère ce rapport
```

Emplacement	Contenu
`frontend/tests/unit/`	TU-01 … TU-06 (Jest + React Testing Library)
`frontend/tests/api/`	TI-01 … TI-07 (Jest, HTTP réel)
`frontend/tests/e2e/`	TS-01 … TS-07 (Playwright)
`frontend/tests/acceptance/` , `qa/acceptance/`	Fiches et script d'aide TA-01 … TA-03
`qa/perf/`	Scénario k6 et résumés
`qa/referentiel.mjs` , `qa/github/`	Référentiel (exigences, cas, anomalies) et publication GitHub
`qa/results/`	Résultats bruts des deux itérations, couvertures, captures, rapports HTML